ICPC TEAMNOTE (Template và Code)

HSG-BANHMI

Mục lục

1 Template thành viên, sinh test **2**
1.1 Template thành viên . . . 2
1.2 Sinh test - Chung . . . 2
1.3 Sinh test - Cây/Đồ thị . . . 3
1.4 Stress Test - So sánh với `brute.cpp` . . . 4
1.5 Stress Test - Hướng dẫn sử dụng . . . 4

2 Toán, công thức **4**
2.1 Hoán vị, chỉnh hợp và tổ hợp . . . 4
2.2 Số Catalan . . . 5
2.3 Số Stirling loại 1 . . . 5
2.4 Số Stirling loại 2 . . . 6
2.5 Lượng giác . . . 6
2.6 Diện tích và thể tích hình cầu . . . 7
2.7 Hệ phương trình bậc nhất hai ẩn . . . 7
2.8 Thuật toán Euclid mở rộng . . . 7

3 Template Code Toán **8**
3.1 Euclid mở rộng . . . 8
3.2 Đặng dư Trung Hoa . . . 8
3.3 Miller Rabin . . . 9

4 Template Code Cấu Trúc Dữ Liệu **9**
4.1 Implicit Treap - Sequence Operations . . . 9

5 Template Code DP **11**
5.1 Bitmask - TSP (Hamiltonion path/cycle . . . 11
5.2 Bitmask - Subset Partition DP . . . 12
5.3 Bitmask - Profile DP (Domino Tiling) . . . 12
5.4 Digit DP - Count Numbers . . . 13
5.5 Tree DP - Rerooting . . . 13
5.6 Divide and Conquer DP Optimization . . . 14
5.7 Convex Hull Trick (CHT) . . . 14

6 Template String **15**
6.1 String Hashing - Double Hash . . . 15

1 Template thành viên, sinh test

1.1 Template thành viên

NQBinh:

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;
using ull = unsigned long long;
using ld = long double;

#define sz(x) ((int)(x).size())
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define hashmap unordered_map
#define hashset unordered_set
#define pb push_back
#define ff first
#define ss second
#define en "\n"

int main()
{
        ios_base::sync_with_stdio(0);
        cin.tie(0);
        cout.tie(0);

        #define task "sample"
        if (fopen(task ".inp", "r"))
        {
                freopen(task ".inp", "r", stdin);
                freopen(task ".out", "w", stdout);
        }



        return 0;
}
```

1.2 Sinh test - Chung

```cpp
#include <bits/stdc++.h>
using namespace std;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

int rnd(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}

long long rndll(long long l, long long r) {
    return uniform_int_distribution<long long>(l, r)(rng);
}

double rndreal(double l, double r) {
    return uniform_real_distribution<double>(l, r)(rng);
}

template<class T>
void shuffleVector(vector<T>& a) {
    shuffle(a.begin(), a.end(), rng);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = rnd(1, 10);
    cout << n << '\n';

    for (int i = 0; i < n; ++i) {
```

```
        cout << rnd(-100, 100) << (i + 1 == n ? '\n' : ' ');
    }

    return 0;
}
```

1.3 Sinh test - Cây/Đồ thị

```
#include <bits/stdc++.h>
using namespace std;

mt19937 rng(chrono::steady_clock::now()
    .time_since_epoch().count());

int rnd(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = rnd(2, 10);
    int type = rnd(0, 2);

    // type = 0: Tree
    // type = 1: Connected graph
    // type = 2: General simple graph

    vector<pair<int, int>> edges;
    set<pair<int, int>> used;

    auto addEdge = [&](int u, int v) {
        if (u > v) swap(u, v);
        if (u == v || used.count({u, v})) return false;

        used.insert({u, v});
        edges.push_back({u, v});
        return true;
    };

    if (type == 0) {
        // Random tree: each vertex connects to an earlier vertex.
        for (int v = 2; v <= n; ++v) {
            int u = rnd(1, v - 1);
            addEdge(u, v);
        }
    } else if (type == 1) {
        // Random connected graph.
        for (int v = 2; v <= n; ++v) {
            int u = rnd(1, v - 1);
            addEdge(u, v);
        }

        int maxM = n * (n - 1) / 2;
        int m = rnd(n - 1, maxM);

        while ((int)edges.size() < m) {
            int u = rnd(1, n);
            int v = rnd(1, n);
            addEdge(u, v);
        }
    } else {
        // General simple graph (possibly disconnected).
        int maxM = n * (n - 1) / 2;
        int m = rnd(0, maxM);

        while ((int)edges.size() < m) {
            int u = rnd(1, n);
            int v = rnd(1, n);
            addEdge(u, v);
        }
    }
```

```
    shuffle(edges.begin(), edges.end(), rng);

    cout << n << ' ' << edges.size() << '\n';

    for (auto [u, v] : edges) {
        cout << u << ' ' << v << '\n';
    }

    return 0;
}
```

1.4 Stress Test - So sánh với `brute.cpp`

```
#!/bin/bash

g++ gen.cpp -o gen -O2
g++ main.cpp -o main -O2
g++ brute.cpp -o brute -O2

for ((tc = 1; ; ++tc)); do
    ./gen > input.txt
    ./main < input.txt > out.txt
    ./brute < input.txt > ans.txt

    if ! diff -w out.txt ans.txt; then
        echo "WA on test $tc"
        cat input.txt
        exit 0
    fi

    echo "Test $tc: OK"
done
```

1.5 Stress Test - Hướng dẫn sử dụng

1. Viết 3 file:
   - `gen.cpp`: Sinh test ngẫu nhiên.
   - `main.cpp`: Code cần kiểm tra.
   - `brute.cpp`: Code vét cạn đúng.
2. Đặt `stress.sh` cùng thư mục với 3 file trên.
3. Chạy stress test:

   ```
   chmod +x stress.sh
   ./stress.sh
   ```

4. Nếu mọi test đúng, chương trình tiếp tục chạy. Nếu sai, chương trình dừng và in ra test gây lỗi.
5. Debug bằng input lỗi, sửa code rồi chạy lại.

Lưu ý:

- `main.cpp` và `brute.cpp` phải cùng format input/output.
- Brute force chỉ nên chạy với test nhỏ.
- Nên kiểm tra thêm các test biên.

2 Toán, công thức

2.1 Hoán vị, chỉnh hợp và tổ hợp

Với $0 \le k \le n$:

$$n! = 1 \cdot 2 \cdot 3 \cdots n, \qquad 0! = 1.$$

Hoán vị:

$$P_n = n!.$$

Chỉnh hợp:

$$A_n^k = \frac{n!}{(n-k)!}.$$

Tổ hợp:

$$\boxed{C_n^k = \binom{n}{k} = \frac{n!}{k!(n-k)!}}$$

Các tính chất:

$$C_n^k = C_n^{n-k}.$$

$$C_n^k = C_{n-1}^{k-1} + C_{n-1}^k.$$

$$\sum_{k=0}^{n} C_n^k = 2^n.$$

$$\sum_{k=0}^{n} kC_n^k = n2^{n-1}.$$

2.2 Số Catalan

$$\boxed{Cat_n = \frac{1}{n+1}\binom{2n}{n}}$$

$$Cat_0 = 1.$$

$$Cat_n = \sum_{i=0}^{n-1} Cat_i Cat_{n-1-i}.$$

$$Cat_n = Cat_{n-1}\frac{4n-2}{n+1}.$$

Một số giá trị đầu:

$$Cat_0 = 1, \quad Cat_1 = 1, \quad Cat_2 = 2,$$
$$Cat_3 = 5, \quad Cat_4 = 14, \quad Cat_5 = 42.$$

2.3 Số Stirling loại 1

Ký hiệu Stirling loại 1 không dấu:

$$\begin{bmatrix} n \\ k \end{bmatrix}.$$

Công thức truy hồi:

$$\boxed{\begin{bmatrix} n \\ k \end{bmatrix} = \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} + (n-1)\begin{bmatrix} n-1 \\ k \end{bmatrix}}$$

Điều kiện biên:

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = 1, \qquad \begin{bmatrix} n \\ 0 \end{bmatrix} = 0 \ (n > 0).$$

$$\begin{bmatrix} n \\ n \end{bmatrix} = 1.$$

Stirling loại 1 có dấu:

$$s(n,k) = s(n-1,k-1) - (n-1)s(n-1,k).$$

2.4 Số Stirling loại 2

Ký hiệu:

$$\left\{ {n \atop k} \right\}.$$

Công thức truy hồi:

$$\boxed{\left\{ {n \atop k} \right\} = \left\{ {n-1 \atop k-1} \right\} + k \left\{ {n-1 \atop k} \right\}}$$

Điều kiện biên:

$$\left\{ {0 \atop 0} \right\} = 1, \qquad \left\{ {n \atop 0} \right\} = 0 \ (n > 0).$$

$$\left\{ {n \atop n} \right\} = 1.$$

2.5 Lượng giác

Đổi độ sang radian:

$$\theta_{\text{rad}} = \theta_{\text{deg}} \frac{\pi}{180}.$$

Đổi radian sang độ:

$$\theta_{\text{deg}} = \theta_{\text{rad}} \frac{180}{\pi}.$$

Các hệ thức cơ bản:

$$\sin^2 x + \cos^2 x = 1.$$

$$\tan x = \frac{\sin x}{\cos x}.$$

$$1 + \tan^2 x = \frac{1}{\cos^2 x}.$$

Công thức cộng:

$$\sin(a+b) = \sin a \cos b + \cos a \sin b.$$

$$\sin(a-b) = \sin a \cos b - \cos a \sin b.$$

$$\cos(a+b) = \cos a \cos b - \sin a \sin b.$$

$$\cos(a-b) = \cos a \cos b + \sin a \sin b.$$

Công thức góc đôi:

$$\sin 2x = 2 \sin x \cos x.$$

$$\cos 2x = 1 - 2\sin^2 x = 2\cos^2 x - 1.$$

Định lý sin:

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R.$$

Định lý cosin:

$$c^2 = a^2 + b^2 - 2ab \cos C.$$

Diện tích tam giác:

$$S = \frac{1}{2} ab \sin C.$$

Công thức Heron:

$$p = \frac{a+b+c}{2}.$$

$$S = \sqrt{p(p-a)(p-b)(p-c)}.$$

2.6 Diện tích và thể tích hình cầu

Với bán kính r và đường kính $d = 2r$:
Diện tích mặt cầu:

$$\boxed{S = 4\pi r^2 = \pi d^2}$$

Thể tích khối cầu:

$$\boxed{V = \frac{4}{3}\pi r^3 = \frac{\pi d^3}{6}}$$

Hằng số:

$$\pi \approx 3.141592653589793.$$

2.7 Hệ phương trình bậc nhất hai ẩn

Xét hệ:

$$\begin{cases} a_1 x + b_1 y = c_1, \\ a_2 x + b_2 y = c_2. \end{cases}$$

Định thức chính:

$$\boxed{D = a_1 b_2 - a_2 b_1}$$

Định thức của x:

$$\boxed{D_x = c_1 b_2 - c_2 b_1}$$

Định thức của y:

$$\boxed{D_y = a_1 c_2 - a_2 c_1}$$

Nếu $D \neq 0$, hệ có nghiệm duy nhất:

$$\boxed{x = \frac{D_x}{D}, \qquad y = \frac{D_y}{D}}$$

Nếu $D = 0$:

- $D_x = D_y = 0$: hệ có vô số nghiệm.
- $D_x \neq 0$ hoặc $D_y \neq 0$: hệ vô nghiệm.

2.8 Thuật toán Euclid mở rộng

Thuật toán tìm x, y thỏa mãn:

$$\boxed{ax + by = \gcd(a, b)}$$

Công thức truy hồi:

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

Nếu:

$$bx' + (a \bmod b)y' = g,$$

thì:

$$\boxed{x = y', \qquad y = x' - \left\lfloor \frac{a}{b} \right\rfloor y'}$$

Phương trình nghiệm nguyên:

$$ax + by = c.$$

Điều kiện có nghiệm nguyên:

$$\boxed{\gcd(a,b) \mid c}$$

Nếu (x_0, y_0) là một nghiệm và $g = \gcd(a,b) > 0$, toàn bộ nghiệm là:

$$\boxed{x = x_0 + \frac{b}{g}t}$$

$$\boxed{y = y_0 - \frac{a}{g}t, \qquad t \in \mathbb{Z}}$$

Nghịch đảo modulo:

$$ax \equiv 1 \pmod{m}.$$

Điều kiện tồn tại:

$$\boxed{\gcd(a,m) = 1}$$

Nếu:

$$ax + my = 1,$$

thì:

$$\boxed{a^{-1} \equiv x \pmod{m}}$$

3 Template Code Toán

3.1 Euclid mở rộng

```
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

ĐPT : $O(log\ min(a,b))$

3.2 Đặng dư Trung Hoa

```
using ll = long long;
using i128 = __int128_t;

ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) {
        x = 1; y = 0;
        return a;
    }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// Returns {x, lcm}; {-1, -1} if no solution.
// Requires positive moduli and all results fit in ll.
pair<ll, ll> CRT(vector<ll> r, vector<ll> m) {
    ll x = 0, mod = 1;
    for (int i = 0; i < (int)r.size(); i++) {
        ll a = r[i], b = m[i];
        a = (a % b + b) % b;
```

```
        ll g = gcd(mod, b);
        ll d = a - x;
        if (d % g) return {-1, -1};

        ll p = b / g, u, v;
        extgcd(mod / g, p, u, v);
        ll t = (ll)(((i128)(d / g) * u % p + p) % p);
        x = (ll)(((i128)x + (i128)mod * t) % ((i128)mod * p));
        mod *= p;
        x = (x % mod + mod) % mod;
    }
    return {x, mod};
}
```

ĐPT : $O(n^2\ log(max\ m_i))$

3.3 Miller Rabin

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 mulmod(u64 a, u64 b, u64 m) {
    return (u128)a * b % m;
}

u64 binpow(u64 a, u64 b, u64 m) {
    u64 r = 1;
    for (; b; b >>= 1, a = mulmod(a, a, m))
        if (b & 1) r = mulmod(r, a, m);
    return r;
}

bool isPrime(u64 n) {
    if (n < 2) return false;
    for (u64 p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n % p == 0) return n == p;
    }

    u64 d = n - 1, s = 0;
    while (!(d & 1)) d >>= 1, ++s;

    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL,
                  450775ULL, 9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
        u64 x = binpow(a % n, d, n);
        if (x == 1 || x == n - 1) continue;

        bool ok = false;
        for (int r = 1; r < s; r++) {
            x = mulmod(x, x, n);
            if (x == n - 1) {
                ok = true;
                break;
            }
        }
        if (!ok) return false;
    }
    return true;
}
```

ĐPT : $O(k\ log^3 n)$

4 Template Code Cấu Trúc Dữ Liệu

4.1 Implicit Treap - Sequence Operations

Ý tưởng: Treap kết hợp BST và Randomized Heap. Không dùng key cố định, vị trí của node được xác định bởi kích thước subtree bên trái.

Hỗ trợ: Insert, Erase, Get, Reverse, Range Query.

ĐPT trung bình: $O(\log n)$ cho mỗi thao tác.

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

struct Node {
    int val, pri, sz;
    bool rev;
    Node *l, *r;

    Node(int v) : val(v), pri(rng()), sz(1), rev(false), l(nullptr), r(nullptr) {}
};

using pNode = Node*;

int size(pNode t) {
    return t ? t->sz : 0;
}

void pull(pNode t) {
    if (t) t->sz = 1 + size(t->l) + size(t->r);
}

void push(pNode t) {
    if (!t || !t->rev) return;

    swap(t->l, t->r);
    if (t->l) t->l->rev ^= 1;
    if (t->r) t->r->rev ^= 1;
    t->rev = false;
}

// Split into first k elements and the rest.
void split(pNode t, int k, pNode &a, pNode &b) {
    if (!t) {
        a = b = nullptr;
        return;
    }

    push(t);

    if (size(t->l) >= k) {
        split(t->l, k, a, t->l);
        b = t;
    } else {
        split(t->r, k - size(t->l) - 1, t->r, b);
        a = t;
    }

    pull(t);
}

// Merge sequences a followed by b.
pNode merge(pNode a, pNode b) {
    if (!a || !b) return a ? a : b;
    if (a->pri > b->pri) {
        push(a);
        a->r = merge(a->r, b);
        pull(a);
        return a;
    } else {
        push(b);
        b->l = merge(a, b->l);
        pull(b);
        return b;
    }
}

// Insert value x at position pos (0-indexed).
void insert(pNode &root, int pos, int x) {
    pNode a, b;
    split(root, pos, a, b);
    root = merge(merge(a, new Node(x)), b);
}

// Erase element at position pos.
void erase(pNode &root, int pos) {
    pNode a, b, c;
    split(root, pos, a, b);
```

```
    split(b, 1, b, c);
    delete b;
    root = merge(a, c);
}

// Reverse interval [l, r), 0-indexed.
void reverseRange(pNode &root, int l, int r) {
    pNode a, b, c;
    split(root, l, a, b);
    split(b, r - l, b, c);
    if (b) b->rev ^= 1;
    root = merge(a, merge(b, c));
}

// Get element at position pos.
int get(pNode root, int pos) {
    push(root);
    int leftSize = size(root->l);
    if (pos < leftSize) return get(root->l, pos);
    if (pos == leftSize) return root->val;
    return get(root->r, pos - leftSize - 1);
}

// Build sequence by appending elements.
void build(pNode &root, const vector<int>& a) {
    root = nullptr;
    for (int x : a) root = merge(root, new Node(x));
}

// Print sequence in order.
void print(pNode t) {
    if (!t) return;

    push(t);
    print(t->l);
    cout << t->val << ' ';
    print(t->r);
}
```

ĐPT trung bình:

- Insert / Erase / Get: $O(\log n)$.
- Reverse đoạn: $O(\log n)$.
- Build bằng merge tuần tự: $O(n \log n)$.
- Bộ nhớ: $O(n)$.

Quy ước:

- Vị trí đánh số từ 0.
- Đoạn $[l, r)$ gồm các phần tử từ l đến $r - 1$.
- $split(t, k, a, b)$ tách k phần tử đầu vào a, phần còn lại vào b.
- Lazy propagation phải được xử lý bằng $push()$ trước khi đi xuống cây.

5 Template Code DP

5.1 Bitmask - TSP (Hamiltonion path/cycle

Trạng thái : $dp[S][u] = \min$ cost

```
const long long INF = (1LL << 60);

int n;
long long cost[20][20];
long long dp[1 << 20][20];

void solve() {
    int FULL = (1 << n) - 1;
    fill(&dp[0][0], &dp[0][0] + (1 << n) * 20, INF);
    dp[1][0] = 0;
    for (int mask = 1; mask <= FULL; ++mask) {
        if (!(mask & 1)) continue;

        for (int u = 0; u < n; ++u) {
```

```
            if (!(mask & (1 << u))) continue;
            if (dp[mask][u] == INF) continue;

            for (int v = 0; v < n; ++v) {
                if (mask & (1 << v)) continue;

                int nxt = mask | (1 << v);
                dp[nxt][v] = min(dp[nxt][v], dp[mask][u] + cost[u][v]);
            }
        }
    }

    long long ans = INF;
    for (int u = 1; u < n; ++u) ans = min(ans, dp[FULL][u] + cost[u][0]);

    cout << ans << '\n';
}
```

ĐPT : $O(n^2 \times 2^n)$

5.2 Bitmask - Subset Partition DP

Trạng thái: $dp[S]$ = chi phí nhỏ nhất để chia tập S thành các nhóm.

```
const long long INF = (1LL << 60);\
int n;
vector<long long> cost, dp;

void solve() {
    int N = 1 << n;
    dp.assign(N, INF);
    dp[0] = 0;
    for (int mask = 1; mask < N; ++mask) {
        int bit = mask & -mask;

        for (int sub = mask; sub; sub = (sub - 1) & mask) {
            if (!(sub & bit)) continue;
            dp[mask] = min(dp[mask], dp[mask ^ sub] + cost[sub]);
        }
    }

    cout << dp[N - 1] << '\\n';
}
```

ĐPT: $O(3^n)$.

5.3 Bitmask - Profile DP (Domino Tiling)

Trạng thái: $dp[col][mask]$ = số cách lát các cột trước đó, với $mask$ mô tả các ô đã được chiếm ở cột hiện tại.

```
const int MOD = 1e9 + 7;
int n, m;
vector<vector<int>> dp;

void solve() {
    int N = 1 << n;
    dp.assign(m + 1, vector<int>(N, 0));
    dp[0][0] = 1;

    for (int col = 0; col < m; ++col) {
        for (int mask = 0; mask < N; ++mask) {
            if (!dp[col][mask]) continue;
            auto dfs = [&](auto&& self, int row, int cur, int nxt) -> void {
                if (row == n) {
                    dp[col + 1][nxt] = (dp[col + 1][nxt] + dp[col][mask]) % MOD;
                    return;
                }

                if (cur & (1 << row)) {
                    self(self, row + 1, cur, nxt);
                    return;
                }
```

```
                if (row + 1 < n && !(cur & (1 << (row + 1)))) {
                    self(self, row + 2, cur | (1 << row) | (1 << (row + 1)), nxt);
                }

                self(self, row + 1, cur | (1 << row), nxt | (1 << row));
            };

            dfs(dfs, 0, mask, 0);
        }
    }

    cout << dp[m][0] << '\\n';
}
```

ĐPT: $O(m \times 2^n \times C)$, với C là số chuyển trạng thái cho mỗi mask; chặn trên thường là $O(2^n)$.

5.4 Digit DP - Count Numbers

Trạng thái: $dp[pos][last][tight][started]$ = số cách điền các chữ số hợp lệ.

Ví dụ: đếm số không có hai chữ số liên tiếp bằng nhau.

```
string s;
long long memo[20][11];
bool vis[20][11];

long long dfs(int pos, int last, bool tight) {
    if (pos == (int)s.size()) return 1;

    if (!tight && vis[pos][last]) return memo[pos][last];

    int lim = tight ? s[pos] - '0' : 9;
    long long ans = 0;
    for (int d = 0; d <= lim; ++d) {
        bool ntight = tight && (d == lim);
        if (d == 0 && last == 10) {
            ans += dfs(pos + 1, 10, ntight);
        } else if (d != last) {
            ans += dfs(pos + 1, d, ntight);
        }
    }

    if (!tight) {
        vis[pos][last] = true;
        memo[pos][last] = ans;
    }

    return ans;
}
```

ĐPT: $O(L \times 11 \times 10)$, với L là số chữ số. Template trên dùng quy ước chữ số 0 ở đầu; cần xử lý riêng số 0 nếu đề không tính số 0 là một số hợp lệ.

5.5 Tree DP - Rerooting

Trạng thái: $sz[u]$ = kích thước subtree của u; $ans[u]$ = tổng khoảng cách từ u đến mọi đỉnh.

```
int n;
vector<vector<int>> adj;
vector<long long> sz, ans;

void dfs1(int u, int p) {
    sz[u] = 1;
    for (int v : adj[u]) {
        if (v == p) continue;
        dfs1(v, u);
        sz[u] += sz[v];
        ans[u] += ans[v] + sz[v];
    }
}

void dfs2(int u, int p) {
    for (int v : adj[u]) {
```

```
        if (v == p) continue;
        ans[v] = ans[u] + n - 2 * sz[v];
        dfs2(v, u);
    }
}

void solve() {
    sz.assign(n, 0);
    ans.assign(n, 0);
    dfs1(0, -1);
    dfs2(0, -1);
}
```

ĐPT: $O(n)$.

5.6 Divide and Conquer DP Optimization

Trạng thái: $dp[i][j] = \min_{k<j}(dp[i-1][k] + cost(k+1, j))$.

Điều kiện: vị trí tối ưu $opt(i, j)$ đơn điệu theo j.

```
const long long INF = (1LL << 60);
vector<long long> prevDP, curDP;

long long cost(int l, int r);

void compute(int L, int R, int optL, int optR) {
    if (L > R) return;
    int mid = (L + R) / 2;
    long long best = INF;
    int opt = -1;
    for (int k = optL; k <= min(mid - 1, optR); ++k) {
        long long val = prevDP[k] + cost(k + 1, mid);
        if (val < best) {
            best = val;
            opt = k;
        }
    }

    curDP[mid] = best;
    if (opt == -1) return;

    compute(L, mid - 1, optL, opt);
    compute(mid + 1, R, opt, optR);
}
```

ĐPT: $O(K \times n \log n)$ nếu mỗi lần tính *cost* mất $O(1)$.

5.7 Convex Hull Trick (CHT)

Trạng thái: $dp[i] = \min_{j<i}(m_j x_i + b_j)$.

Dùng khi mỗi trạng thái trước tương ứng với một đường thẳng và cần truy vấn min/max tại x_i.

```
struct Line {
    long long m, b;

    long long get(long long x) const {
        return m * x + b;
    }
};

struct CHT {
    deque<Line> hull;

    void add(long long m, long long b) {
        Line nw = {m, b};

        while (hull.size() >= 2) {
            Line a = hull[hull.size() - 2];
            Line b = hull.back();
            __int128 lhs = (__int128)(b.b - a.b) * (b.m - nw.m);
            __int128 rhs = (__int128)(nw.b - b.b) * (a.m - b.m);
            if (lhs >= rhs) hull.pop_back();
```

```
            else break;
        }

        hull.push_back(nw);
    }

    long long query(long long x) {
        while (hull.size() >= 2 && hull[0].get(x) >= hull[1].get(x)) {
            hull.pop_front();
        }

        return hull.front().get(x);
    }
};
```

ĐPT: $O(n)$ khi slope và truy vấn có thứ tự phù hợp. Cần điều chỉnh điều kiện loại đường thẳng tùy quy ước slope tăng/giảm và bài toán min/max.

6 Template String

6.1 String Hashing - Double Hash

Ý tưởng: Ánh xạ một chuỗi thành giá trị hash để so sánh substring trong $O(1)$ sau khi tiền xử lý.

```
using ull = unsigned long long;
const int MOD1 = 1e9 + 7;
const int MOD2 = 1e9 + 9;
const int BASE = 311;

struct Hashing {
    int n;
    vector<ull> h1, h2, p1, p2;

    Hashing(string s) {
        n = s.size();
        h1.assign(n + 1, 0);
        h2.assign(n + 1, 0);
        p1.assign(n + 1, 1);
        p2.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            int c = s[i] - 'a' + 1;
            h1[i + 1] = (h1[i] * BASE + c) % MOD1;
            h2[i + 1] = (h2[i] * BASE + c) % MOD2;
            p1[i + 1] = p1[i] * BASE % MOD1;
            p2[i + 1] = p2[i] * BASE % MOD2;
        }
    }

    pair<ull, ull> get(int l, int r) {
        // Hash substring [l, r), 0-indexed
        ull x = (h1[r] - h1[l] * p1[r-l] % MOD1 + MOD1) % MOD1;
        ull y = (h2[r] - h2[l] * p2[r-l] % MOD2 + MOD2) % MOD2;
        return {x, y};
    }

    bool equal(int l1, int r1, int l2, int r2) {
        if (r1 - l1 != r2 - l2) return false;
        return get(l1, r1) == get(l2, r2);
    }
};
```

ĐPT:

- Preprocessing: $O(n)$.
- Hash substring: $O(1)$.
- So sánh hai substring: $O(1)$.
- Bộ nhớ: $O(n)$.

Ứng dụng:

- So sánh hai substring.
- Kiểm tra palindrome kết hợp hash đảo ngược.

- Đếm substring khác nhau.
- Tìm chu kỳ và phát hiện chuỗi lặp.

Lưu ý:

- Đoạn substring dùng quy ước $[l, r)$.
- Template trên giả sử chuỗi chỉ gồm chữ cái thường.
- Double hash giảm mạnh xác suất collision nhưng không loại bỏ hoàn toàn.